CYLIXLAB · STUDY GUIDE

Module 01

Foundations of AI & Machine Learning Algorithms

Theory | Supervised · Unsupervised · Optimization (GD, Adam) · Evaluation (Precision, Recall, F1, ROC-AUC)

What you will master

Section	Topic	You should be able to…
1	Foundations of AI & ML	Place ML in the AI landscape, name the learning paradigms and explain generalization
2	Supervised Learning	Choose and justify regression / classification models, diagnose over- & underfitting
3	Unsupervised Learning	Apply clustering, PCA and anomaly detection and judge cluster quality
4	Optimization	Derive and apply Gradient Descent, Momentum and Adam updates by hand
5	Evaluation Metrics	Compute and interpret Precision, Recall, F1, ROC-AUC and pick the right one
6	Revision Kit	Use the formula sheet and pitfall list before the MCQ assessment

KEY IDEA
Learning in one sentence: a machine-learning algorithm improves its performance on a task by minimizing a **loss** measured on data, using an **optimizer**, and is judged by how well it **generalizes** to unseen data, using suitable **metrics**. Every section of this module is one piece of that sentence.

1 Foundations of AI & Machine Learning

Artificial Intelligence (AI) is the broad goal of building systems that perform tasks requiring human-like intelligence. **Machine Learning (ML)** is the subset of AI where behaviour is *learned from data* instead of hand-coded as rules. **Deep Learning (DL)** is the subset of ML that uses multi-layer neural networks to learn hierarchical representations.

1.1 Learning paradigms

Paradigm	Data you have	Goal	Typical examples
Supervised	Inputs **x** with labels **y**	Learn mapping f(x) → y	Spam detection, house-price prediction
Unsupervised	Inputs **x** only	Discover structure	Customer segmentation, PCA, anomaly detection
Semi-supervised	Few labels + many unlabelled	Use unlabelled data to boost accuracy	Medical imaging with few expert labels
Self-supervised	Labels created from the data itself	Learn representations	Next-word prediction, masked image modelling
Reinforcement	Agent, environment, rewards	Maximize cumulative reward	Game playing, robotics, recommendation policies

1.2 The standard ML workflow

Define the problem → **collect & clean data** → **split** (train / validation / test) → **engineer features** → **choose a model & loss** → **optimize** → **evaluate** → **tune & deploy**. Skipping or reordering these steps (especially splitting) is the most common source of misleading results.

1.3 Generalization, bias and variance

A model is useful only if it performs well on **unseen** data. The expected error of a model decomposes into three parts:

$$\text{Error}_{\text{test}} = \text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$$

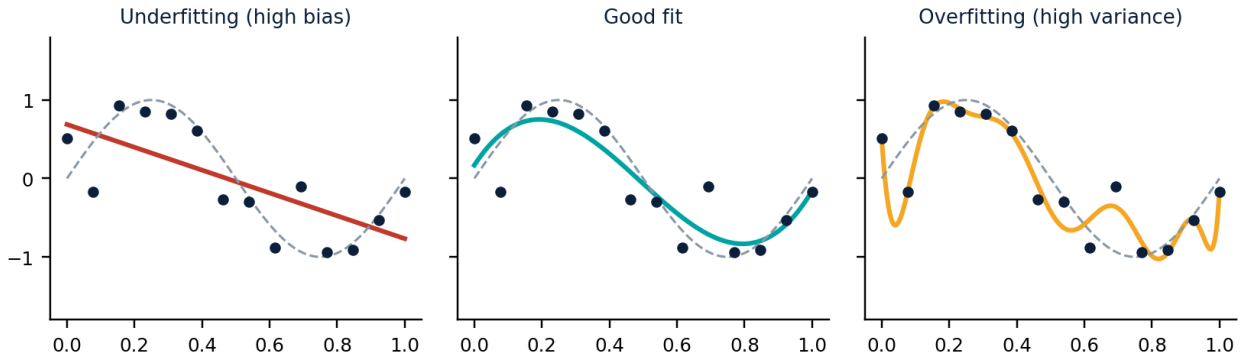


Figure 1: Same data, three model complexities. Dashed line = true function.

	Underfitting (high bias)	Overfitting (high variance)
Training error	High	Very low
Validation error	High (≈ training error)	Much higher than training error
Cause	Model too simple, too few features, over-regularized	Model too complex, too little data, noisy labels
Fixes	More capacity, better features, less regularization	More data, regularization, early stopping, simpler model, dropout

1.4 Data splitting, cross-validation and leakage

- **Train** set fits the parameters; **validation** set tunes hyper-parameters and picks models; **test** set is touched *once* for the final unbiased estimate.
- **k-fold cross-validation** splits the training data into k folds; each fold is used once for validation. Report the mean ± standard deviation. Use **stratified** folds for imbalanced classes and **time-ordered** splits for time series.
- **Data leakage** is when information from outside the training set influences the model (e.g. fitting a scaler on the whole dataset before splitting, or random-shuffling time-series data). It inflates scores and fails in production.

COMMON PITFALL
Fit every preprocessing step (scaler, imputer, encoder, PCA) on the **training data only**, then apply it to validation/test data. Wrap them in a pipeline so cross-validation does this automatically.

2 Supervised Learning

Given a dataset $\{(x_i, y_i)\}$, the model f_θ learns parameters θ that minimize a loss L between predictions and labels. If y is **continuous** the task is **regression**; if y is a **category** it is **classification**.

2.1 Linear regression

Predicts a continuous target as a weighted sum of features, trained by minimizing the Mean Squared Error (MSE):

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b, \qquad L(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

It has a closed-form solution (the **normal equation**) $\mathbf{w} = (X^\top X)^{-1} X^\top y$, but gradient descent is preferred when the number of features or samples is large. Key assumptions: linear relationship, independent errors, roughly constant error variance, and limited multicollinearity.

2.2 Logistic regression (classification)

Despite the name, logistic regression is a **classifier**. It squashes a linear score z through the sigmoid to give a probability, and is trained with the binary cross-entropy (log-loss):

$$P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \mathbf{w}^\top \mathbf{x} + b$$

$$L = -\frac{1}{n}\sum_{i=1}^{n} [\, y_i \log p_i + (1 - y_i) \log(1 - p_i) \,]$$

The output is a *probability*; the class is decided by a **decision threshold** (default 0.5). Moving the threshold trades precision against recall (Section 5). For K classes, use the **softmax** function with categorical cross-entropy.

2.3 Common supervised algorithms

Algorithm	How it works	Strengths	Watch out for
k-Nearest Neighbours	Predicts from the k closest training points	Simple, non-parametric, no training	Needs **feature scaling**; slow at prediction; curse of dimensionality
Naive Bayes	Bayes' theorem + feature independence	Fast, works with little data, good for text	Independence assumption rarely true
Decision Tree	Recursive splits that reduce impurity (Gini / entropy)	Interpretable, no scaling needed	Overfits when deep; unstable
Random Forest	Many trees on bootstrap samples; votes/averages	Robust, strong baseline, handles mixed data	Less interpretable; large models
Gradient Boosting	Trees added sequentially to fix previous errors	Top accuracy on tabular data	Sensitive to hyper-parameters; can overfit
SVM	Maximum-margin hyperplane (kernels for non-linearity)	Effective in high dimensions	Slow on huge data; needs scaling and tuning
Neural Network	Layers of weighted sums + non-linear activations	Learns complex patterns (images, text)	Needs data, compute, careful tuning

2.4 Regularization: controlling model complexity

Technique	Penalty / idea	Effect
L2 (Ridge)	$+ \lambda \Sigma w_j^2$	Shrinks all weights smoothly; helps with multicollinearity
L1 (Lasso)	$+ \lambda \Sigma \lvert w_j \rvert$	Drives some weights to **exactly zero**: built-in **feature selection**
Elastic Net	Mix of L1 and L2	Sparsity plus stability with correlated features
Early stopping / Dropout	Stop training when validation loss rises / randomly drop units	Prevents co-adaptation and memorization in neural networks

KEY IDEA
Larger λ ⇒ simpler model ⇒ lower variance but higher bias. Pick λ by cross-validation, never by looking at the test set.

EXAM TIP
Scenario questions often hide the answer in the data description: *continuous target* → regression; *many irrelevant features + want sparsity* → L1; *distance-based model with mixed units* → scale features.

3 Unsupervised Learning

Unsupervised algorithms find structure in **unlabelled** data. The three main families are **clustering** (grouping similar points), **dimensionality reduction** (compressing features) and **anomaly detection** (finding unusual points).

3.1 K-means clustering

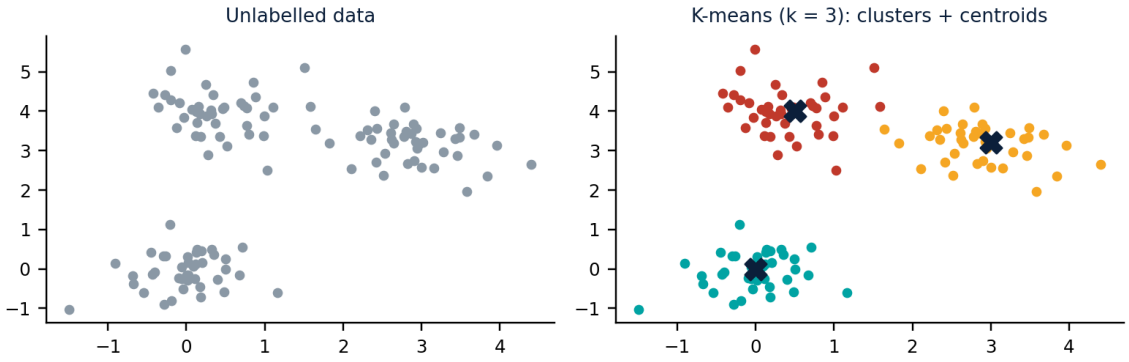


Figure 2: K-means groups unlabelled points around k centroids.

K-means minimizes the **within-cluster sum of squares** (inertia):

$$J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

- **1. Initialize** k centroids (use **k-means++** for smarter seeds).
- **2. Assign** every point to its nearest centroid.
- **3. Update** each centroid to the mean of its points.
- **4. Repeat** steps 2–3 until assignments stop changing (convergence).

K-means assumes roughly **spherical, similar-sized** clusters, is sensitive to initialization (run several restarts, *n_init*) and to feature scale, and requires you to choose k.

Choosing k

- **Elbow method:** plot inertia vs k and choose the “elbow” where the improvement drops sharply.
- **Silhouette score** for a point: s = (b − a) / max(a, b), where a = mean distance to its own cluster and b = mean distance to the nearest other cluster. Range −1 to +1; near +1 = well clustered, near 0 = on a boundary, negative = probably misassigned.

3.2 Other clustering methods

Method	Idea	Use when	Limits
Hierarchical (agglomerative)	Merge the closest clusters step by step → dendrogram	You want a hierarchy; k unknown	$O(n^2)$ memory/time
DBSCAN	Dense regions (ε radius, minPts) form clusters; sparse points = **noise**	Arbitrary shapes (e.g. crescents), outliers present	Struggles with varying density; tune ε
Gaussian Mixture (GMM)	Soft assignment with probabilistic Gaussians (EM)	Overlapping, elliptical clusters	Assumes Gaussian shapes

3.3 Dimensionality reduction: PCA

Principal Component Analysis projects data onto orthogonal directions (principal components) of **maximum variance**. Steps: standardize → compute the covariance matrix → eigen-decomposition →

keep the top-m eigenvectors → project.

$$\text{Explained variance ratio}_j = \frac{\lambda_j}{\sum_i \lambda_i}$$

- Choose m so the cumulative explained variance reaches a target (e.g. **95%**).
- PCA is **linear** and sensitive to scale; always **standardize** first.
- Use **t-SNE / UMAP** only for 2-D/3-D visualization of non-linear structure, not as general preprocessing.

3.4 Anomaly detection

Detects rare points that differ from the majority (fraud, faults, intrusions) without needing many labelled anomalies. Common approaches: **Isolation Forest** (anomalies are isolated in few random splits), **One-Class SVM**, **density-based** (DBSCAN noise, LOF) and **reconstruction error** (autoencoders, PCA).

COMMON PITFALL
Unsupervised results have no ground-truth accuracy. Validate with internal metrics (silhouette, inertia), stability across runs, and, most importantly, whether the clusters are **useful** for the business question.

4 Optimization: Gradient Descent and Adam

Training means finding parameters θ that minimize the loss L(θ). Because L is rarely solvable in closed form, we use **iterative gradient-based optimization**: move θ a small step *opposite* to the gradient, which points uphill.

4.1 Gradient Descent (GD)

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

η (eta) is the **learning rate**, the single most important hyper-parameter. Too small → painfully slow; too large → overshoots, oscillates or diverges.

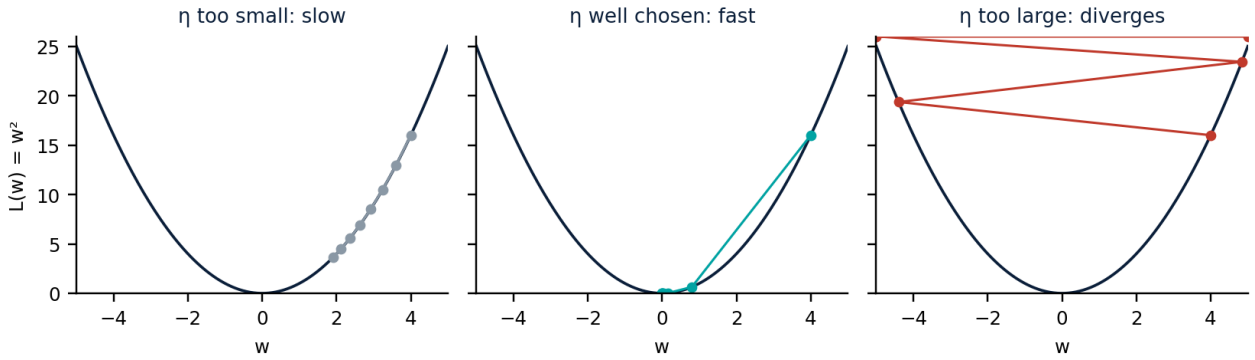


Figure 3: Effect of the learning rate on L(w) = w² starting from w = 4.

WORKED EXAMPLE
L(w) = w^2, so ∇L = 2w. Start at w_0 = 4 with η = 0.1.
Step 1: w_1 = 4 − 0.1·(2·4) = **3.2**. Step 2: w_2 = 3.2 − 0.1·(6.4) = **2.56**. Each step multiplies w by (1 − 2η) = 0.8, so w shrinks smoothly toward the minimum at 0. With η = 1.05 the factor is 1 − 2.1 = −1.1, so |w| *grows* and training diverges.

4.2 Batch, stochastic and mini-batch GD

Variant	Gradient computed on	Pros	Cons
Batch GD	All n samples	Stable, smooth convergence	Slow and memory-heavy for large n
Stochastic GD (SGD)	1 sample	Very fast updates; noise can escape shallow minima	Very noisy path
Mini-batch GD	Batch of B (e.g. 32–512)	Best trade-off; GPU-friendly; **default in deep learning**	Batch size is another hyper-parameter

4.3 Momentum and RMSProp: the building blocks of Adam

Momentum keeps an exponentially-weighted average of past gradients (a “velocity”). It accelerates movement along consistent directions and **damps oscillations** across steep ravines.

$$v_t = \beta\, v_{t-1} + \nabla L(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta\, v_t$$

RMSProp divides each step by the root of a running average of *squared* gradients, giving every parameter its own effective learning rate: small steps where gradients are large, bigger steps where they are tiny or sparse.

$$s_t = \rho\, s_{t-1} + (1-\rho)\, g_t^2, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t} + \varepsilon}\, g_t$$

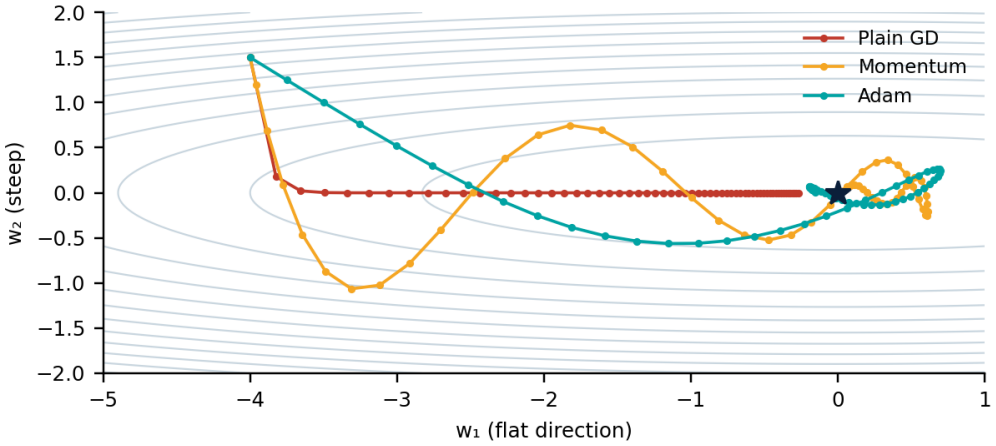


Figure 4: On an elongated loss surface, plain GD zig-zags; momentum and Adam reach the minimum faster.

4.4 Adam (Adaptive Moment Estimation)

Adam combines **momentum** (first moment m) with **RMSProp** (second moment v) and adds **bias correction**. With gradient $g_t = \nabla L(\theta_t)$:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

Symbol	Meaning	Typical default
η	Learning rate	10^{-3} (0.001)
β_1	Decay of first moment (momentum)	0.9
β_2	Decay of second moment (squared gradients)	0.999
ε	Numerical stability constant	10^{-8}

- **Why bias correction?** m and v start at 0, so early estimates are biased toward zero. Dividing by $(1 - \beta^t)$ removes this bias; its effect fades as t grows.
- **Per-parameter step sizes:** parameters with consistently large gradients get smaller steps; rare/sparse features get larger ones, which is why Adam works well for embeddings and NLP.
- **AdamW** decouples weight decay from the gradient update. Adding L2 loss inside Adam is *not* equivalent to true weight decay, so AdamW usually generalizes better.
- **Practice:** pair any optimizer with a learning-rate **schedule** (warm-up, step or cosine decay) and consider **gradient clipping** for unstable training.

WORKED EXAMPLE
First Adam step with $g_1 = 2$, $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ ($m_0 = v_0 = 0$):
$m_1 = 0.1 \cdot 2 = 0.2 \rightarrow \hat{m}_1 = 0.2/0.1 = \mathbf{2}$. $v_1 = 0.001 \cdot 4 = 0.004 \rightarrow \hat{v}_1 = 0.004/0.001 = \mathbf{4}$.
Step $= 0.001 \cdot 2/(\sqrt{4} + \varepsilon) \approx \mathbf{0.001}$. On the very first step Adam moves each parameter by about η in the direction of −sign(g), regardless of the gradient's magnitude.

Optimizer	Best for	Notes
SGD + Momentum	Vision models; best final generalization when tuned	Needs careful LR schedule
RMSProp	Recurrent networks, non-stationary problems	Adaptive per-parameter rates
Adam / AdamW	Default choice; transformers, sparse features, quick prototyping	Robust to LR choice; AdamW preferred with weight decay

5 Evaluation Metrics for Classification

Choosing the metric is a **business decision**: it must reflect the real cost of each type of mistake. All threshold-based metrics derive from the **confusion matrix**.

5.1 Confusion matrix

	Predicted Positive	Predicted Negative
Actual Positive	**TP**: true positive (hit)	**FN**: false negative (miss, Type II)
Actual Negative	**FP**: false positive (false alarm, Type I)	**TN**: true negative (correct rejection)

Metric	Formula	Answers the question…
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall, how often is the model right?
Precision	TP / (TP + FP)	Of everything predicted positive, how much was truly positive?
Recall (Sensitivity, TPR)	TP / (TP + FN)	Of all real positives, how many did we catch?
Specificity (TNR)	TN / (TN + FP)	Of all real negatives, how many did we correctly clear?
False Positive Rate	FP / (FP + TN) = 1 − Specificity	How many negatives raised a false alarm?
F1 score	2PR / (P + R)	Single score balancing precision and recall

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\,TP}{2\,TP + FP + FN}, \qquad F_\beta = (1+\beta^2)\frac{P \cdot R}{\beta^2 P + R}$$

F1 is the **harmonic mean** of precision and recall, so it is dragged down by whichever is smaller (P = 0.9, R = 0.5 gives F1 ≈ 0.64, not 0.70). Use $\mathbf{F_\beta}$ with β > 1 to weight recall more (e.g. F2) and β < 1 to weight precision more.

WORKED EXAMPLE
TP = 40, FP = 10, FN = 20, TN = 130 (n = 200).
Accuracy = 170/200 = **0.85**. Precision = 40/50 = **0.80**. Recall = 40/60 = **0.667**. Specificity = 130/140 = **0.929**.
F1 = 2(0.80)(0.667)/(0.80 + 0.667) = **0.727**.

5.2 Which metric when?

Situation	Costly mistake	Prioritize
Cancer / disease screening, safety faults	Missing a real case (FN)	**Recall** (lower the threshold)
Spam filter, content moderation auto-ban	Flagging a legitimate item (FP)	**Precision** (raise the threshold)
Imbalanced data, both errors matter	Either	**F1** / PR-AUC
Balanced classes, equal costs	Either	Accuracy is acceptable

COMMON PITFALL
The accuracy paradox. If only 1% of transactions are fraud, a model that always predicts “not fraud” scores 99% accuracy but has **0% recall**. Never trust accuracy alone on imbalanced data.

5.3 Threshold trade-off, ROC and AUC

A probabilistic classifier outputs scores; the threshold turns them into labels. **Lowering** the threshold catches more positives (recall ↑) but raises false alarms (precision usually ↓, FPR ↑); **raising** it does the reverse. The **ROC curve** plots TPR against FPR across *all* thresholds.

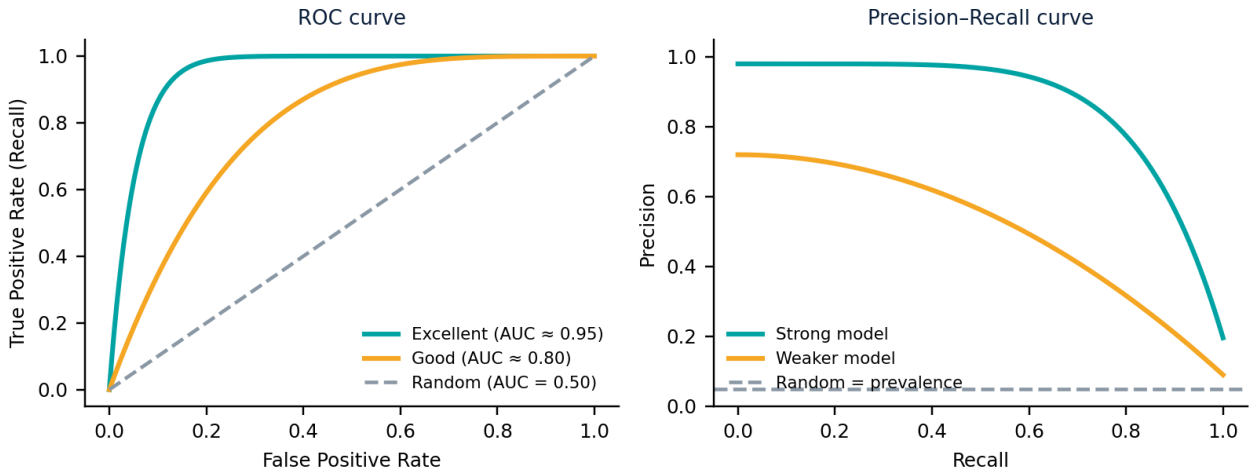


Figure 5: Left: ROC curves. Right: Precision–Recall curves, more informative on rare positive classes.

- **AUC (Area Under the ROC Curve)** equals the probability that the model ranks a random positive higher than a random negative.
- **AUC = 0.5** → no better than random guessing; **1.0** → perfect ranking; **< 0.5** → predictions are systematically inverted (flip them).
- AUC is **threshold-independent** and insensitive to class prior, which makes it great for comparing models but can look **over-optimistic when positives are very rare**, because the huge number of true negatives keeps FPR tiny.
- For heavy imbalance, prefer the **Precision–Recall curve** and **Average Precision (PR-AUC)**; a random classifier's PR-AUC equals the positive-class prevalence.

5.4 Multi-class averaging and regression metrics

Averaging	How	Use when
Macro	Compute metric per class, then take the plain average	Every class matters equally, including rare ones
Micro	Pool all TP/FP/FN across classes, then compute	You care about overall instance-level performance
Weighted	Per-class average weighted by class support	Imbalanced classes, but reflect frequency

Regression metric	Formula	Notes
MAE	$(1/n)\ \Sigma\ \lvert y - \hat{y}\rvert$	Robust to outliers, same units as y
RMSE	$\sqrt{[(1/n)\ \Sigma\ (y - \hat{y})^2]}$	Penalizes large errors heavily
R^2	$1 - SS_{res} / SS_{tot}$	Fraction of variance explained; can be negative on test data

6 Revision Kit

6.1 One-page formula sheet

Concept	Formula / rule
MSE	$(1/n)\ \Sigma\ (y - \hat{y})^2$
Sigmoid	$\sigma(z) = 1 / (1 + e^{-z})$
Log-loss	$-[y \log p + (1 - y) \log(1 - p)]$
K-means objective	$\Sigma_k\ \Sigma_{x \in C_k} \|x - \mu_k\|^2$
Silhouette	$s = (b - a) / \max(a, b)$
Gradient descent	$\theta \leftarrow \theta - \eta \nabla L$
Momentum	$v \leftarrow \beta v + g;\ \theta \leftarrow \theta - \eta v$
Adam	$m \leftarrow \beta_1 m + (1-\beta_1) g;\ v \leftarrow \beta_2 v + (1-\beta_2) g^2;\ \theta \leftarrow \theta - \eta\ \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$
Precision / Recall	TP/(TP+FP) / TP/(TP+FN)
F1	2PR/(P+R)
FPR / TPR	FP/(FP+TN) / TP/(TP+FN)

6.2 Ten classic mistakes

- Fitting scalers/PCA on the full dataset before splitting (leakage).
- Using accuracy on imbalanced data.
- Using k-means or kNN without scaling features.
- Choosing k, λ or η using the test set.
- Reading ROC-AUC as “accuracy”, or trusting it blindly at extreme imbalance.
- Averaging precision and recall arithmetically instead of using the harmonic mean (F1).
- Shuffling time-series data randomly.
- Assuming a lower training loss always means a better model.
- Forgetting that Adam’s default η = 0.001 may still need a schedule.
- Treating cluster labels as ground truth.

6.3 Self-check before the MCQ assessment

- Can you say why L1 yields sparse weights while L2 does not?
- Can you compute silhouette, precision, recall and F1 from raw numbers?
- Can you perform two steps of GD and one step of Adam by hand?
- Can you justify *which metric* to optimize for a given business scenario?
- Can you explain what AUC = 0.5 and AUC = 0.95 mean in plain language?

KEY IDEA
Next step: attempt the **Module 01 MCQ Assessment** (30 scenario-based questions). For every wrong answer, re-read the matching section and write one sentence explaining *why* your choice was tempting. That is where the learning happens.